

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flappy Bird</title>
  <!-- Import sound library -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/tone/14.8.49/Tone.js"></script
>

  <style>
    body {
      margin: 0;
      padding: 0;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      height: 100vh;
      background-color: #2c3e50; /* Dark blue-gray background */
      color: white;
      font-family: 'Arial', sans-serif;
      overflow: hidden; /* Hide scrollbars */
    }
    canvas {
      /* Background is drawn in JS, so we can have layers */
      background-color: transparent;
      border: 5px solid #2c3e50; /* Dark border */
      border-radius: 10px;
      display: block;
      max-width: 90%;
      max-height: 90%;
      box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2);
    }
    #game-container {
      position: relative;

```

```
        display: flex;
        justify-content: center;
        align-items: center;
    }
    #ui-container {
        position: absolute;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
        display: flex;
        flex-direction: column;
        justify-content: center;
        align-items: center;
        pointer-events: none; /* Allow clicks to pass through to canvas
*/
    }
    #score {
        font-size: 4em;
        font-weight: bold;
        text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
        z-index: 10;
        position: absolute;
        top: 20px;
        display: none; /* Hidden until game starts */
    }
    #start-message {
        display: flex;
        flex-direction: column;
        justify-content: center;
        align-items: center;
        font-size: 1.5em;
        text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
        text-align: center;
        z-index: 20;
    }
}
```

```

#start-message h1 {
    font-size: 2.5em;
    margin: 0;
}

#game-over-message {
    display: none; /* Hidden by default */
    flex-direction: column;
    justify-content: center;
    align-items: center;
    background-color: rgba(44, 62, 80, 0.85);
    padding: 30px;
    border-radius: 15px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.3);
    text-align: center;
    z-index: 20;
    pointer-events: all; /* Enable pointer events for this element
*/
}

#game-over-message h1 {
    margin: 0 0 10px 0;
    font-size: 3em;
    color: #e74c3c; /* Red color for 'Game Over' */
}

#game-over-message p {
    font-size: 1.5em;
    margin: 10px 0;
}

#restart-button {
    font-size: 1.2em;
    padding: 12px 25px;
    background-color: #2ecc71; /* Green button */
    border: none;
    border-radius: 8px;
    color: white;
    font-weight: bold;
}

```

```

        cursor: pointer;
        box-shadow: 0 4px 0 #27ae60; /* Button shadow */
        transition: all 0.1s ease;
    }
    #restart-button:active {
        transform: translateY(2px);
        box-shadow: 0 2px 0 #27ae60;
    }
</style>
</head>
<body>

    <div id="game-container">
        <canvas id="gameCanvas" width="400" height="600"></canvas>
        <div id="ui-container">
            <div id="start-message">
                <h1>Flappy Bird</h1>
                <p>Click or Press Space to Start</p>
            </div>
            <div id="score">0</div>
            <div id="game-over-message">
                <h1>Game Over</h1>
                <p>Your Score: <span id="final-score">0</span></p>
                <button id="restart-button">Restart</button>
            </div>
        </div>
    </div>

    <script>
        // Get canvas and context
        const canvas = document.getElementById('gameCanvas');
        const ctx = canvas.getContext('2d');

        // Get UI elements
        const scoreDisplay = document.getElementById('score');
        const startMessage = document.getElementById('start-message');

```

```

    const gameOverMessage =
document.getElementById('game-over-message');

    const finalScoreDisplay = document.getElementById('final-score');
    const restartButton = document.getElementById('restart-button');

    // Game variables
    let bird, pipes, score, gravity, lift;
    let isGameOver = false;
    let isGameStarted = false;
    let frameCount = 0;
    let clouds = [];

    // --- Sound Setup ---
    let sounds = {};
    function initSounds() {
        sounds.flap = new Tone.Synth({
            oscillator: { type: 'sine' },
            envelope: { attack: 0.01, decay: 0.1, sustain: 0, release:
0.1 }

        }).toDestination();

        sounds.score = new Tone.Synth({
            oscillator: { type: 'triangle' },
            envelope: { attack: 0.01, decay: 0.1, sustain: 0, release:
0.1 }

        }).toDestination();

        sounds.crash = new Tone.NoiseSynth({
            noise: { type: 'white' },
            envelope: { attack: 0.01, decay: 0.3, sustain: 0, release:
0.1 }

        }).toDestination();
    }

    // Bird properties
    const birdProps = {

```

```

    x: 80,
    y: canvas.height / 2,
    radius: 15,
    velocity: 0,
    rotation: 0
  };

  // Pipe properties
  const pipeProps = {
    width: 70,
    gap: 160,
    spawnInterval: 180, // frames between pipes
    speed: 3
  };

  // --- Background ---
  function createCloud() {
    clouds.push({
      x: canvas.width,
      y: Math.random() * (canvas.height - 100) + 50,
      radius: Math.random() * 20 + 20,
      speed: Math.random() * 0.5 + 0.2 // Slower than pipes
    });
  }

  function drawBackground() {
    // Draw sky
    ctx.fillStyle = '#70c5ce'; // Sky blue
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    // Spawn new clouds
    if (frameCount % 200 === 0) {
      createCloud();
    }

    // Draw and move clouds

```

```

    ctx.fillStyle = 'rgba(255, 255, 255, 0.8)';
    for (let i = clouds.length - 1; i >= 0; i--) {
        let cloud = clouds[i];
        if (isGameStarted && !isGameOver) {
            cloud.x -= cloud.speed;
        }

        // Draw cloud as a series of circles
        ctx.beginPath();
        ctx.arc(cloud.x, cloud.y, cloud.radius, 0, Math.PI * 2);
        ctx.arc(cloud.x + cloud.radius * 0.8, cloud.y, cloud.radius
* 0.8, 0, Math.PI * 2);
        ctx.arc(cloud.x - cloud.radius * 0.8, cloud.y, cloud.radius
* 0.7, 0, Math.PI * 2);
        ctx.fill();

        // Remove off-screen clouds
        if (cloud.x + cloud.radius * 2 < 0) {
            clouds.splice(i, 1);
        }
    }
}

// --- Bird ---
function drawBird() {
    ctx.save();
    // Translate and rotate context to bird's position
    ctx.translate(bird.x, bird.y);
    ctx.rotate(bird.rotation);

    // Draw bird body (yellow circle)
    ctx.fillStyle = '#f1c40f'; // Yellow
    ctx.beginPath();
    ctx.arc(0, 0, bird.radius, 0, Math.PI * 2);
    ctx.fill();

```

```

        // Draw eye
        ctx.fillStyle = 'white';
        ctx.beginPath();
        ctx.arc(bird.radius * 0.3, -bird.radius * 0.4, bird.radius *
0.3, 0, Math.PI * 2);
        ctx.fill();
        ctx.fillStyle = 'black';
        ctx.beginPath();
        ctx.arc(bird.radius * 0.4, -bird.radius * 0.4, bird.radius *
0.15, 0, Math.PI * 2);
        ctx.fill();

        // Draw beak
        ctx.fillStyle = '#e67e22'; // Orange
        ctx.beginPath();
        ctx.moveTo(bird.radius * 0.5, 0);
        ctx.lineTo(bird.radius * 1.2, -bird.radius * 0.1);
        ctx.lineTo(bird.radius * 1.2, bird.radius * 0.1);
        ctx.closePath();
        ctx.fill();

        ctx.restore(); // Restore context to original state
    }

    function updateBird() {
        bird.velocity += gravity;
        bird.y += bird.velocity;

        // Update rotation
        if (bird.velocity > 0.5) { // Falling
            bird.rotation += 0.05;
            if (bird.rotation > Math.PI / 2) {
                bird.rotation = Math.PI / 2; // Cap at 90 deg
            }
        }
    }

```



```

        // Check for boundaries
        if (bird.y + bird.radius > canvas.height) {
            bird.y = canvas.height - bird.radius;
            bird.velocity = 0;
            endGame();
        }
        if (bird.y - bird.radius < 0) {
            bird.y = 0;
            bird.velocity = 0;
            endGame();
        }
    }

    function flap() {
        if (isGameOver) return;

        // Start the game on first flap
        if (!isGameStarted) {
            startGame();
        }

        // Play flap sound
        if (sounds.flap) {
            sounds.flap.triggerAttackRelease("C5", "8n");
        }

        bird.velocity = lift;
        bird.rotation = -0.5; // Rotate up (~-30 deg)
    }

    // --- Pipes ---
    function drawPipes() {
        ctx.fillStyle = '#2ecc71'; // Green color
        ctx.strokeStyle = '#27ae60'; // Darker green
        ctx.lineWidth = 4;
    }

```

```

    for (let pipe of pipes) {
        // Top pipe
        ctx.fillRect(pipe.x, 0, pipe.width, pipe.topHeight);
        ctx.strokeRect(pipe.x, 0, pipe.width, pipe.topHeight);
        // Top pipe cap
        ctx.fillRect(pipe.x - 5, pipe.topHeight - 20, pipe.width +
10, 20);

        ctx.strokeRect(pipe.x - 5, pipe.topHeight - 20, pipe.width
+ 10, 20);


        // Bottom pipe
        ctx.fillRect(pipe.x, pipe.bottomY, pipe.width,
pipe.bottomHeight);
        ctx.strokeRect(pipe.x, pipe.bottomY, pipe.width,
pipe.bottomHeight);
        // Bottom pipe cap
        ctx.fillRect(pipe.x - 5, pipe.bottomY, pipe.width + 10,
20);

        ctx.strokeRect(pipe.x - 5, pipe.bottomY, pipe.width + 10,
20);
    }
}

function updatePipes() {
    // Spawn new pipe
    if (frameCount % pipeProps.spawnInterval === 0) {
        const topHeight = Math.random() * (canvas.height / 2 -
pipeProps.gap / 2) + 60;
        const bottomY = topHeight + pipeProps.gap;
        pipes.push({
            x: canvas.width,
            width: pipeProps.width,
            topHeight: topHeight,
            bottomY: bottomY,
            bottomHeight: canvas.height - bottomY,

```

```

        passed: false
    });
}

// Move pipes
for (let i = pipes.length - 1; i >= 0; i--) {
    let pipe = pipes[i];
    pipe.x -= pipeProps.speed;

    // Collision detection
    if (
        bird.x + bird.radius > pipe.x &&
        bird.x - bird.radius < pipe.x + pipe.width &&
        (bird.y - bird.radius < pipe.topHeight || bird.y +
bird.radius > pipe.bottomY)
    ) {
        endGame();
    }

    // Update score
    if (pipe.x + pipe.width < bird.x && !pipe.passed) {
        pipe.passed = true;
        score++;
        scoreDisplay.textContent = score;
        if (sounds.score) {
            sounds.score.triggerAttackRelease("E5", "8n");
        }
    }

    // Remove off-screen pipes
    if (pipe.x + pipe.width < 0) {
        pipes.splice(i, 1);
    }
}
}

```

```
// --- Game Logic ---

function resetGame() {
    bird = { ...birdProps, y: canvas.height / 2 };
    pipes = [];
    clouds = [];
    score = 0;
    gravity = 0.5;
    lift = -10;
    isGameOver = false;
    isGameStarted = false;
    frameCount = 0;

    scoreDisplay.textContent = '0';
    gameOverMessage.style.display = 'none';
    startMessage.style.display = 'flex';
    scoreDisplay.style.display = 'none';
}

function startGame() {
    isGameStarted = true;
    startMessage.style.display = 'none';
    scoreDisplay.style.display = 'block';
}

function endGame() {
    if (isGameOver) return; // Prevent multiple calls
    isGameOver = true;
    if (sounds.crash) {
        sounds.crash.triggerAttackRelease("0.2");
    }
    finalScoreDisplay.textContent = score;
    gameOverMessage.style.display = 'flex';
    scoreDisplay.style.display = 'none';
}

function gameLoop() {
```

```

    // Clear canvas
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // Draw background
    drawBackground();

    // Draw pipes first (behind bird)
    drawPipes();

    if (isGameOver) {
        // If game is over, just draw the bird in its final spot
        drawBird();
    } else if (!isGameStarted) {
        // On start screen, make bird hover
        bird.y = birdProps.y + Math.sin(frameCount * 0.05) * 5;
        drawBird();
    } else {
        // If game is running
        updateBird();
        drawBird();
        updatePipes();
    }

    if (!isGameOver) {
        frameCount++;
    }

    requestAnimationFrame(gameLoop);
}

// --- Event Listeners ---
function handleInput() {
    // Initialize sounds on first user interaction
    if (Tone.context.state !== 'running') {
        Tone.start();
        initSounds();
    }
}

```

```

    }
    flap();
}

window.addEventListener('keydown', (e) => {
    if (e.code === 'Space') {
        handleInput();
    }
});

canvas.addEventListener('mousedown', handleInput);
canvas.addEventListener('touchstart', (e) => {
    e.preventDefault(); // Prevent screen zoom
    handleInput();
}, { passive: false });

restartButton.addEventListener('click', (e) => {
    e.stopPropagation(); // Prevent click from bubbling to canvas
    resetGame();
});

// --- Responsive Canvas ---
function resizeCanvas() {
    const container = document.getElementById('game-container');
    const aspectRatio = 400 / 600;

    let newWidth = window.innerWidth * 0.9;
    let newHeight = window.innerHeight * 0.9;

    if (newWidth / newHeight > aspectRatio) {
        newWidth = newHeight * aspectRatio;
    } else {
        newHeight = newWidth / aspectRatio;
    }
}

```

```
// Limit max size
if (newWidth > 400) {
    newWidth = 400;
    newHeight = 600;
}

canvas.width = newWidth;
canvas.height = newHeight;

// Adjust bird and pipe properties based on new size
birdProps.y = canvas.height / 2;
pipeProps.gap = newHeight * 0.27; // 160 / 600

// Re-center UI
const scoreTop = 20 * (newHeight / 600);
scoreDisplay.style.top = `${scoreTop}px`;
scoreDisplay.style.fontSize = `${4 * (newHeight / 600)}em`;

// Restart game on resize to reset positions
resetGame();
}

// Initial setup
resizeCanvas();
window.addEventListener('resize', resizeCanvas);

// Start the game loop (it will show start screen first)
gameLoop();

</script>
</body>
</html>
```